NAME: Harsh Singh

REGISTRATION NO.: 25BCE10783

AI-Based Study Weakness Analyzer

Bring Your Own Project (BYOP) Report

1. Introduction

This Study Weakness Analyzer is a Python-based project developed to bridge the gap between basic digital participation and true digital literacy. Instead of a generic schedule, the code uses custom logic to prioritize subjects where performance is low but difficulty is high, turning raw academic data into an actionable roadmap. By inputting current marks and a difficulty rating (1–5), the program automatically generates a personalized 6-hour daily study plan and predicts potential score boosts. To maintain a humanized, zero-dependency design, I implemented a text-based bar chart using ASCII symbols to visualize performance without needing external libraries. Ultimately, this project demonstrates how to use technology responsibly to make informed decisions, transforming it from a simple convenience into a meaningful tool for academic growth.

2. Problem Statement

The core problem highlighted in this study is the "illusion of digital competence," where students mistake basic participation for true digital literacy. While most are comfortable navigating user-friendly platforms and creating polished content, this familiarity is often limited to basic functionality rather than a deep understanding of digital systems, data privacy, or online risks. Because modern tools are designed to be intuitive and automated, they reduce the need for critical thinking, leading students to prioritize how their work looks over what it actually represents. This gap in awareness results in poor decision-making, such as misrepresenting skills on professional platforms like LinkedIn or becoming vulnerable to cyber threats driven by convenience and urgency. Ultimately, without shifting from simple participation to actual literacy, technology remains a source of potential risk rather than a tool for meaningful growth.

3. Objective

The primary objective of this study is to bridge the gap between "digital participation"—simply using technology—and true "digital literacy," which involves a deep, responsible understanding of digital systems. While many students appear tech-savvy because they can navigate user-friendly platforms, they often lack critical awareness regarding data privacy, ethical content usage, and online security. This case study aims to highlight how the "illusion of competence" created by intuitive tools can lead to risky behaviors, such as misrepresenting skills on professional profiles or falling victim to cyber threats. Ultimately, the goal is to encourage students to move beyond surface-level usage and develop the habit of critical thinking and accountability in every digital interaction.

4. Proposed Solution

The proposed solution focuses on bridging the gap between basic digital participation and true digital literacy by shifting from passive usage to active, responsible decision-making. It suggests that students must move beyond the "illusion of competence" created by user-friendly, automated tools and instead develop critical thinking habits, such as verifying information before acting on it. This involves taking personal accountability for one's digital footprint—ensuring authenticity on professional platforms like LinkedIn and maintaining formal etiquette in academic communication. Furthermore, the solution emphasizes that cyber safety should be treated as a continuous behavioral practice of "pausing and questioning" rather than just a one-time lesson. Ultimately, the goal is to transform technology from a mere convenience into a meaningful tool for growth by consistently applying ethical and security knowledge in real-world situations.

5.SOURCE CODE

```python
# This version predicts improvement based on study hours

print("----- STUDY WEAKNESS ANALYZER -----\n")

# Just using simple lists for everything
names = []
marks = []
diffs = []

# Ask for total subjects
n = int(input("How many subjects do you have? "))

# Input loop
for i in range(n):
    print("\nSubject " + str(i+1))
    s_name = input("Name: ")
    s_marks = float(input("Marks (out of 100): "))
    s_diff = int(input("Difficulty (1-5): "))

names.append(s_name)
marks.append(s_marks)
diffs.append(s_diff)

# Logic for study hours: (100 - marks) * difficulty / 50
# Basically, lower marks + high difficulty = more time
temp_h = []
for i in range(n):
    val = (100 - marks[i]) * diffs[i] / 50
temp_h.append(val)

# Normalize to a 6-hour study day
total_temp = sum(temp_h)
final_h = []
for h in temp_h:
    # Scale it so the total is exactly 6 hours
    final_h.append((h / total_temp) * 6)

print("\n" + "="*30)
print("ANALYSIS RESULT")
print("="*30)

# Finding the weakest subject manually
low_score = 101
worst_sub = ""
```

```python
# Finding the weakest subject manually
low_score = 101
worst_sub = ""

for i in range(n):
# Manual classification
  if marks[i] < 50:
     lab = "Weak"
  elif marks[i] < 70:
     lab = "Average"
  else:
     lab = "Strong"

# Track the lowest one
if marks[i] < low_score:
    low_score = marks[i]
    worst_sub = names[i]
print(names[i] + " | Marks: " + str(marks[i]) + " | Status: " + lab)

print("\n ALERT: You really need to focus on " + worst_sub.upper())

print("\n----- PREDICTED MARKS BOOST -----")
# Logic: Every hour of study adds 2 marks to the score
for i in range(n):
    boost = marks[i] + (final_h[i] * 2)
# Cap it at 100 max
if boost > 100:
    boost = 100
    print(names[i] + ": Now " + str(marks[i]) + " -> After Study: " + str(round(boost, 1)))

# Making a text-based bar chart using symbols
print("\n----- PERFORMANCE GRAPH -----")
for i in range(n):
 # One block for every 10 marks
  blocks = "▇" * int(marks[i] // 10)
print(names[i].ljust(10) + ": " + blocks + " (" + str(marks[i]) + ")")

print("\n----- DAILY STUDY PLAN -----")
for i in range(n):
   print("- " + names[i] + ": " + str(round(final_h[i], 2)) + " hours")

print("\nDone! Follow this and you'll improve for sure.")
```

6. Methodology

The methodology follows a simple three-step logic: **Data Input**, **Weighted Calculation**, and **Visual Reporting**. First, it collects subject names, marks, and a 1–5 difficulty rating to establish a performance baseline. Second, it applies a normalization formula that weighs low scores against high difficulty, scaling the total study time into a fixed 6-hour daily window. Finally, the program categorizes subjects by status and generates a text-based ASCII bar chart to visualize the results. This manual approach using core Python lists and loops demonstrates a move from basic digital participation to data-driven literacy.

7. Tools and Technologies Used

This project was built entirely using **Python 3.x**, focusing on core programming logic rather than external libraries to demonstrate fundamental digital literacy. The primary tools used include **dynamic lists** for data storage, **`for` loops** for iterative calculations, and **`if-elif-else` statements** for performance categorization. For the mathematical component, I implemented a **normalization algorithm** to scale study hours into a fixed 6-hour window and used the **`round()` function** for clean output. The visual interface relies on **ASCII string manipulation** to create a text-based bar chart ( ) and **CLI (Command Line Interface)** interactions for user input and real-time alerts.

8. Implementation and Result

The project is implemented as a **Python-based CLI application** that processes user inputs for subject performance and difficulty. It uses an iterative loop to collect data, which is then stored in dynamic lists for calculation. The core result is a **data-driven study roadmap** that normalizes study time into a fixed 6-hour window. The output provides a clear categorization of subjects (Strong, Average, Weak), a predictive analysis of potential score improvements, and a text-based ASCII bar chart to visualize academic standing without requiring external libraries.

9. Challenges Faced

A major challenge was developing a **fair time-distribution algorithm** that prioritized both low scores and high difficulty simultaneously while capping the total output at exactly 6 hours. I had to manually handle the math to ensure the proportions remained realistic. Additionally, creating a **visual performance graph** using only standard string manipulation ( ) was a hurdle, as I wanted the tool to be lightweight and functional in any basic Python environment without needing complex installations like Matplotlib.

10. Future Enhancements

In the future, I plan to evolve this script into a **full-stack web application** using frameworks like Flask or Django. This would allow for **persistent data storage** via a SQL database, enabling students to track their progress over time. I also aim to integrate the **Google Calendar API** to sync the generated study plans directly to mobile devices and add a "Pomodoro Timer" feature to help users maintain focus during their allocated study blocks.

11. Conclusion

This project demonstrates how logical programming can solve the real-world problem of academic burnout and poor prioritization. By moving away from the **illusion of digital competence**, the Study Weakness Analyzer provides a roadmap that forces students to confront their weak spots. It proves that technology is most effective when used as a tool

for **accountability and structured growth**, ensuring that preparation is based on actual academic needs rather than just convenience.

12. Learning **Outcomes**

Building this tool has significantly improved my **algorithmic thinking** and my ability to translate abstract problems into functional code. I learned how to handle **data normalization** and manual list processing, which are foundational skills in software development. Most importantly, the project taught me the value of **digital literacy**—understanding that the true power of a tool lies in how accurately it reflects reality and helps a user make informed, responsible decisions.